NourishNet

Final Report

DC Data Challenge 2026

Team Name:

Data4Good [University of Maryland]

Team Members:

Harsh Shrishrimal

Jiten Bhalavat

Rishabh Ranka

Rishika Thakre

Sahil Chordia

April 13, 2026

Intended Users and How the Tool Supports Them

We built NourishNet for three groups of people: **Donors**, **Volunteers**, and **individuals and families in need**. Each group has a different connection to food insecurity in Maryland and the DC metro area.

Consumers: Families and Individuals in Need

When hungry and in need of help, the last thing people require is searching across multiple websites or navigating PDFs to find assistance. That is why we created the **Find Food** webpage: users can enter their ZIP code, address, or city name and immediately see food pantries, food banks, SNAP authorized retailers, and farmers' markets available nearby on an interactive map.

Users can sort results based on when they need food, which county they are in, and what type of service they prefer. Each result displays its address, a clickable phone number, working hours, and one-tap directions in Google Maps. We also included an **"I Need Food Now"** button that instantly surfaces the three most accessible food sources and the 211 hotline number. All functionality is accessible without signing in or providing any personal data.

Donors

Many people want to donate food or money but do not know where to start, or want to help the areas that need it most. Our **Donate** page displays more than 1,440 donation locations on a map alongside a food insecurity layer that highlights which areas have greater need.

When a donor hovers over a county, they can see the county name and the number of food-insecure residents: data sourced from Feeding America. We also built a **chatbot powered by Google Gemini**. Donors can ask questions such as where to donate near a given ZIP code and receive nearby locations with addresses and phone numbers. The chatbot reads from a dataset of thousands of records and uses keyword matching to surface relevant results.

Volunteers

The **Volunteer** page displays opportunities connected to food banks and pantries. Users can filter by county and by weekday or weekend. Each opportunity includes contact details and schedule information, along with an interest form that opens the user's email client with a pre-written message to the organization. No user data is stored on our servers.

User Interface Features and Flow

The app has five main pages connected by a navigation bar at the top.

Home Page

The Home page presents four main options: *Find Food*, *Donate*, *Community*, and *About*: and displays summary statistics, including **505 organizations**, **4,196 SNAP locations**,

and **382 food banks**.

Find Food Page

Users enter a location, which is geocoded using **Nominatim** from OpenStreetMap. If the location matches a city or region, its boundary is drawn on the map and results inside it are displayed. A side panel shows results sorted by distance; clicking a result highlights it on the map and reveals full details. Filters allow narrowing by type, county, and day. A **"Use My Location"** button uses the browser's geolocation to show nearby places.

Donate Page

Similar to Find Food, this page adds a **food insecurity map layer** using color shading from light orange to dark red to indicate need intensity. Users can toggle the layer on or off. Donation location cards include website, email, and instructions. A chatbot button opens a chat window for questions.

Volunteer Page

Volunteer opportunities are listed with schedules and filterable by county and availability. Each listing includes a button that opens a pre-addressed email to express interest.

About Page

The About page includes an image, information cards explaining the project, and a disclaimer with the 211 hotline number.

Responsive Design

The app is fully responsive. On mobile devices, the map occupies approximately half the screen with the results list below, filters arranged vertically, and larger map icons for easier tapping.

Backend Data Process and Architecture

Our data pipeline consists of four stages and is written in **Python**.

Stage 1: Data Collection

We collected data from more than **25 sources**:

- **Active scraping:** SNAP store data via the USDA ArcGIS API.
- **Passive collection:** Food pantry listings and PDF files.
- **Direct downloads:** Feeding America statistics and farmers market data as CSV files.

All sources are listed in a configuration file.

Stage 2: Data Cleaning

- Removed data outside Maryland and DC.
- Removed duplicate entries based on name and address.
- Merged similar datasets and removed unnecessary fields.

Stage 3: Normalization

- Cleaned formatting issues such as extra whitespace.
- Standardized phone numbers into a single format.
- Converted currency and percentage values to proper numeric types.
- Made county names consistent across all sources.
- Filled missing values with sensible defaults.

Stage 4: Unification

Data was organized into three folders for consumer, donor, and volunteer use. Column names were standardized and text was formatted consistently.

Geocoding

A geocoding script added latitude and longitude to locations that were missing coordinates, using the **Google Maps Geocoding API** with a local cache file to avoid repeated requests.

System Architecture

Output Files

The pipeline produces two JSON files:

1. **Consumer data:** Food locations and volunteer opportunities.
2. **Donor data:** Donation locations and food insecurity statistics.

Each record includes fields such as `id`, `type`, `name`, `address`, `coordinates`, `phone`, `hours`, and `tags`. Donor records additionally include donation types and payment methods. County records include food insecurity rates and population figures.

Chatbot Backend

The chatbot backend is built using **FastAPI**. It loads all data at startup. When a user submits a question, the system searches the dataset using keyword matching and selects the most relevant results. These results are passed to **Google Gemini** alongside the question, with a system prompt designed to keep responses short and accurate.

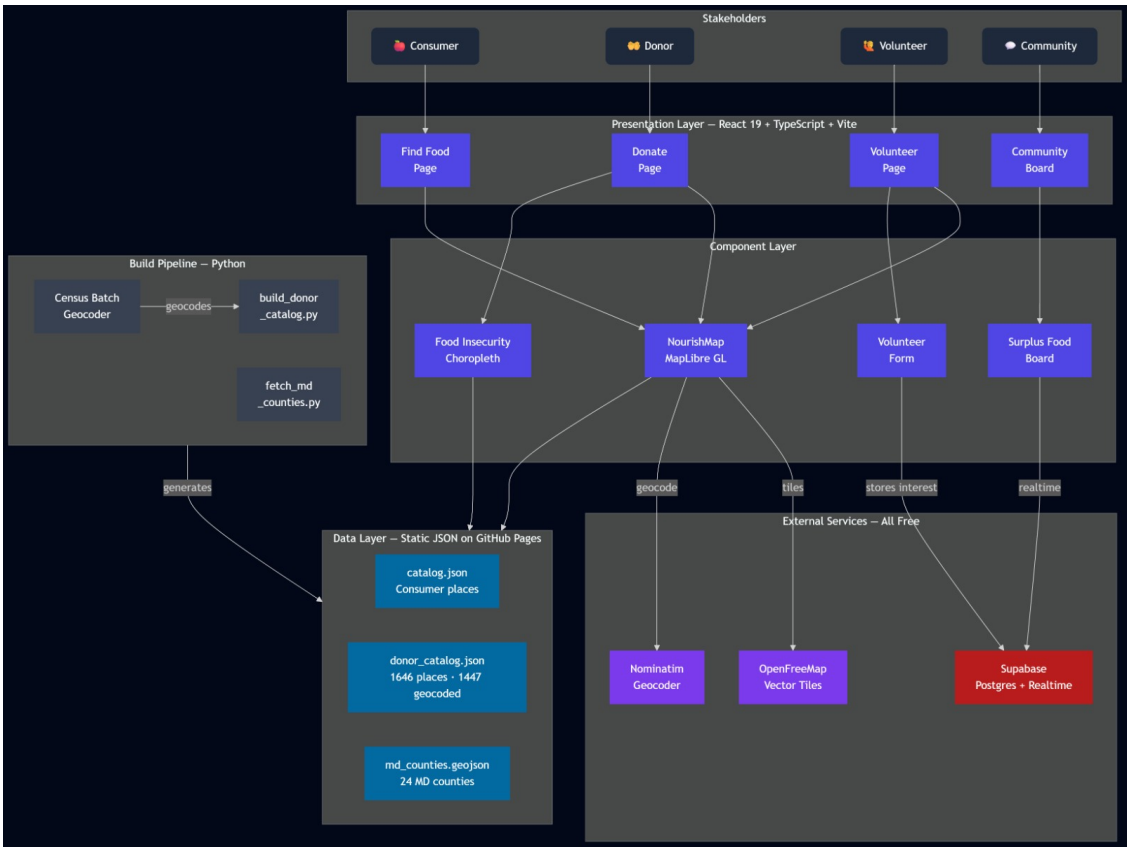

Figure 1: NourishNet System Architecture

Additional Data Sources

Beyond the provided data, we used the following public sources:

- **USDA SNAP Retailer Locator:** SNAP store locations
- **Feeding America:** Food insecurity statistics
- **US Census Bureau:** Local area demographic data
- **Maryland GIS:** Map boundary files
- **211 Maryland:** Food pantry listings
- **Capital Area Food Bank:** Partner organization locations
- **Caroline Better Together:** Pantry data extracted from PDFs
- **Prince George's County Open Data:** Farmers markets and food store data

Experience Using Prompt Engineering

We used **Kiro** as our primary development tool, storing all prompts in a dedicated file. Our process followed a step-by-step approach: the first prompt established the full app structure: pages, maps, and sample data: giving us a functional starting point.

Tasks were divided among the five team members and all prompts were recorded. Key lessons learned:

- **Specificity matters:** Clear, detailed instructions (exact colors, exact features) produced correct results immediately; vague instructions required many iterations.
- **Context improves output:** Providing Kiro with access to documentation and data structure files helped it correctly wire data to the application.
- **Hybrid approach:** The chatbot was partly generated by Kiro and partly improved manually, particularly the system prompt and keyword-matching logic.

Future Improvements

Given additional time, we would pursue the following enhancements:

a) **Real-time data:** Scheduled scripts to regularly collect and refresh data automatically.

b) **SMS integration:** Allow users to text a number and receive nearby food location information.

c) **Public transportation integration:** Surface transit directions to assist users without personal vehicles.

d) **Multilingual support:** Expand language coverage and enable the chatbot to respond in multiple languages.

e) **Feedback system:** Allow users to rate results and report issues to continuously improve the platform.

Changes Made to Generated Code

Most of the React frontend was produced by Kiro through our prompt engineering workflow. The main areas where we made substantial manual changes:

- **Data pipeline:** The entire Python ETL pipeline was written collaboratively with Kiro but required significant manual debugging, especially around path issues, encoding problems, and edge cases in the raw data (malformed CSV rows, inconsistent county names, mixed encodings).
- **Chatbot system prompt:** Kiro generated the FastAPI structure, but we manually wrote and iterated on the system prompt over several rounds of testing to get the response quality right. The keyword scoring logic was also manually tuned.

- **Food insecurity map:** Kiro produced the initial MapLibre choropleth code, but we manually fixed county name normalization (the GeoJSON uses "Prince George's" while Feeding America data uses "Prince George's County") and adjusted the color scale to avoid white buckets that were invisible on the map..
- **Geocoding cache:** The geocode_catalog.js script was written to batch-geocode 1,200+ pantry addresses. We added a local JSON cache to avoid re-geocoding addresses we had already looked up, which saved significant API quota.
- **About page:** The cascading card animation and overall About page design went through many iterations with Kiro, with us providing very specific visual direction for colors, layout, and animation behavior.

Conclusion

NourishNet brings together data from dozens of sources into a single, accessible platform to help people find food, donate, and volunteer. The system is designed to be simple, privacy-respecting, and usable by everyone: whether they are searching for a meal tonight or looking to support their community.